Clase 18 · Chat grupal con select() y sockets crudos

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 27 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

El Programa 07 (chat grupal con select()) línea por línea, prueba en vivo con nc, el bloque /16 de PC Puma e introducción al sniffer con sockets crudos. Continúa la Clase 17.

> **Fuente**
>
> NotebookLM (audio ACS-Clase18.m4a). Lo que NotebookLM agregó sin contrastar con el audio se marca como dudoso.

1. El Programa 07: un solo proceso con select()

Servidor TCP en C que atiende a muchos clientes desde **un solo proceso** con multiplexación de E/S, y reenvía cada mensaje a todos los demás (chat grupal). Analogía: el operador de radio de una flota de taxis, que repite a todos lo que dice uno.

Preparación

1. `client_socket[max_clients]` lleno de ceros: 0 = lugar libre.
2. `socket(AF_INET, SOCK_STREAM, 0)`.
3. `setsockopt` con `SO_REUSEADDR`: reutilizar el puerto de inmediato al relanzar.
4. `sockaddr_in`: `AF_INET`, `INADDR_ANY` (todas las interfaces), puerto de la línea de comandos con `htons()`.
5. `bind()` y `listen()` con backlog de 10.

El ciclo while(1)

1. `FD_ZERO(&readfds)` vacía el conjunto.
2. `FD_SET(master_socket, &readfds)` mete el socket servidor.
3. Cada `client_socket[i] > 0` entra con `FD_SET`; se calcula `max_fd`.
4. `select(max_fd + 1, &readfds, NULL, NULL, NULL)`: duerme sin gastar CPU hasta que hay actividad.
5. Actividad en el socket servidor → conexión nueva: `accept()`, imprime "Cliente conectado" con IP y puerto efímero, guarda el descriptor en el primer lugar == 0.
6. Actividad de un cliente (`FD_ISSET`) → `read()`:
   - 0: "Cliente desconectado", `close()`, su lugar vuelve a 0.
   - > 0: arma el texto con IP y puerto del remitente y hace `send()` a todos los lugares ocupados **excepto el remitente**.

Parámetros de select() (Open Group)

Parámetro	Qué es
nfds	Rango 0 a nfds − 1; por eso max_fd + 1
readfds	Conjunto a vigilar para lectura (el único que usa el Programa 07)
writefds	Conjunto para escritura (NULL)
errorfds	Conjunto para errores (NULL)
timeout	struct timeval con el tiempo máximo; NULL = esperar indefinidamente

Macros de fd_set

Macro	Hace
FD_ZERO(&set)	Vacía el conjunto
FD_SET(fd, &set)	Agrega un descriptor
FD_CLR(fd, &set)	Quita un descriptor
FD_ISSET(fd, &set)	¿Ese descriptor tiene actividad?

Por qué select() y no fork()

Los hijos de fork() tienen memoria aislada y no pueden pasarse mensajes (Clase 17). Con un solo proceso, el arreglo client_socket está a la vista de todo el código.

Error común: no repetir FD_ZERO y el llenado en cada vuelta; select() modifica readfds y deja solo los descriptores con actividad.

2. Prueba en vivo con nc

```
gcc programa_07_servidor_grupal.c -o programa_07
./programa_07 4898            # servidor 132.248.59.6
nc -v 132.248.59.6 4898       # cada alumno
```

- Desde la red inalámbrica de la UNAM todos salían con la misma IP pública, `132.247.100.226`, pero cada uno con un **puerto efímero distinto**.
- **Sin eco:** el servidor no le devuelve el mensaje a quien lo escribió.
- **Sin palabra de salida:** se sale con `Ctrl + C`; `read()` devuelve 0 y aparece "Cliente desconectado".
- **Sin contador de usuarios:** el profesor lo propuso como mejora.

Error común: creer que la IP identifica a cada alumno; los distingue el puerto efímero.

3. PC Puma: un bloque /16

Consultando `132.247.100.226` en ipinfo.io: bloque /16 de la UNAM. 32 − 16 = 16 bits → 2^16 = **65,536 direcciones**.

4. Sniffer con sockets crudos (herramienta extra)

```
int sock_raw = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

- **Capa 2** (enlace): el kernel entrega la trama Ethernet completa, sin quitarle las capas como con SOCK_STREAM.
- `ETH_P_ALL`: todos los protocolos.
- Requiere **root** (`sudo`); con usuario normal `socket()` falla por permisos.
- **Modo promiscuo**: ve tráfico que no va a la MAC propia.
- Si un servidor manda texto sin cifrar (como el chat del Programa 07), el sniffer lo lee tal cual.

Dudoso: NotebookLM menciona de pasada una "regla de las 4 P de la seguridad" que no pudo reconstruir.

Pendiente

- Sin entregas nuevas (tareas 1 a 4 ya calificadas, según NotebookLM).
- Calificaciones del parcial por correo hasta el fin de semana; quien quedó en la bitácora del servidor remoto y mandó su código la tiene asegurada.
- Tarea moral: probar el Programa 07 en casa y revisar en GitHub el código del sniffer.
- Jueves 29 oct: sniffer con sockets crudos en vivo.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Idea para recordar
select()	Un proceso vigila muchos descriptores y duerme sin gastar CPU
Vuelta del ciclo	FD_ZERO → FD_SET servidor y clientes → select(max_fd + 1) → FD_ISSET
read() = 0	Cliente desconectado: close() y lugar a 0
read() > 0	send() a todos menos al remitente
nfds	max_fd + 1 porque revisa de 0 a nfds − 1
Misma IP pública	La distingue el puerto efímero
/16	2^16 = 65,536 direcciones
Socket crudo	AF_PACKET + SOCK_RAW, capa 2, root, modo promiscuo

Hilo conductor

fork() aísla la memoria de cada cliente; select() los junta en un solo proceso para reenviar mensajes. Y lo que viaja sin cifrar lo puede leer cualquiera con un socket crudo.